

"""

fase3_multivariado.py

=====

FASE 3 - Pruebas multivariadas: redundancia y colinealidad.

Las fases 1 y 2 juzgan cada variable en soledad. Dos variables pueden ser excelentes por separado y, aun asi, ser la MISMA informacion. Mantener ambas:

- * infla la varianza de los coeficientes en modelos lineales / logisticos (los signos se vuelven inestables y la interpretacion deja de ser fiable),
- * reparte la importancia entre clones en modelos de arboles, escondiendo la senal real,
- * encarece el proceso productivo sin ganancia de desempeno.

Procedimiento

- 3.1 Matriz de asociacion en [0, 1] homogenea para tipos mixtos (|Spearman| numerica-numerica, V de Cramer categorica-categorica, razon de correlacion eta numerica-categorica).
- 3.2 Regla de exclusion por redundancia: para cada par con asociacion mayor al umbral, GANA la variable con mayor ``score\_compuesto`` de la fase 2, es decir la de mayor poder predictivo combinado (Gini + IV). Los pares se procesan de mayor a menor asociacion, de modo que las redundancias mas flagrantes se resuelven primero.
- 3.3 Complemento: VIF, que detecta colinealidad MULTIPLE (una variable explicada por una combinacion de otras sin estar muy correlacionada con ninguna en particular). Por defecto solo informa.

"""

from __future__ import annotations

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger
from .metricas import calcular_vif, matriz_asociacion

LOGGER = obtener_logger("fase3")

```
def _tipo_par(a: str, b: str, tipos: dict[str, str]) -> str:  
    """Describe que medida de asociacion se aplico a un par."""  
    num = ("NUMERICA", "BOOLEANA")  
    ta, tb = tipos.get(a, "CATEGORICA"), tipos.get(b, "CATEGORICA")  
    if ta in num and tb in num:  
        return "|correlacion| num-num"  
    if ta not in num and tb not in num:  
        return "V de Cramer cat-cat"  
    return "razon de correlacion (eta) num-cat"
```

```
def ejecutar(  
    df: pd.DataFrame,  
    cfg: ConfigPipeline,  
    tipos: dict[str, str],  
    sobrevivientes: list[str],  
    biv: pd.DataFrame,  
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:  
    """Ejecuta la fase multivariada.
```

Returns

(tabla_multivariada, matriz_asociacion, pares_redundantes)

"""

LOGGER.info("=" * 78)

LOGGER.info("FASE 3 - PRUEBAS MULTIVARIADAS (REDUNDANCIA Y COLINEALIDAD)")

LOGGER.info(

```

        "Variables=%d | umbral asociacion=%.2f | metodo num-num=%s | umbral VIF=%.1f",
        len(sobrevivientes), cfg.umbral_correlacion, cfg.metodo_correlacion, cfg.umbral_vif,
    )
    LOGGER.info("=" * 78)

    vacio = pd.DataFrame()
    if len(sobrevivientes) == 0:
        LOGGER.warning("No hay variables que evaluar en la fase multivariada.")
        return vacio, vacio, vacio

    # Puntajes heredados de la fase 2 (criterio de desempate).
    scores = biv.set_index("columna")[["iv", "gini", "score_compuesto"]].to_dict("index")

    if len(sobrevivientes) == 1:
        col = sobrevivientes[0]
        info = scores.get(col, {})
        multi = pd.DataFrame([
            {
                "columna": col, "correlacion_maxima": np.nan, "variable_correlacionada": "",
                "tipo_asociacion": "", "iv": info.get("iv", np.nan), "gini": info.get("gini", np.nan),
                "score_compuesto": info.get("score_compuesto", np.nan), "vif": np.nan,
                "n_pares_redundantes": 0, "flg_exclusion_multivariada": 0, "flg_vif_alto": 0,
                "flg_seleccion_final": 1, "motivo_exclusion_multivariada": "",
                "decision_multivariada": "RETENIDA (unica variable superviviente)",
            }
        ])
        return multi, vacio, vacio

    # -----
    # 3.1 Matriz de asociacion
    # -----
    LOGGER.info("Calculando matriz de asociacion (%d x %d)...", len(sobrevivientes), len(sobrevivientes))
    M = matriz_asociacion(df[sobrevivientes], tipos, cfg.metodo_correlacion)

    # -----
    # Pares por encima del umbral, ordenados de mas a menos redundante
    # -----
    pares: list[dict] = []
    cols = list(M.columns)
    for i, a in enumerate(cols):
        for b in cols[i + 1:]:
            valor = float(M.loc[a, b])
            if np.isfinite(valor) and valor > cfg.umbral_correlacion:
                sa = scores.get(a, {}).get("score_compuesto", np.nan)
                sb = scores.get(b, {}).get("score_compuesto", np.nan)
                pares.append({
                    "variable_a": a, "variable_b": b, "asociacion": valor,
                    "tipo_asociacion": _tipo_par(a, b, tipos),
                    "score_a": sa, "score_b": sb,
                    "iv_a": scores.get(a, {}).get("iv", np.nan),
                    "iv_b": scores.get(b, {}).get("iv", np.nan),
                    "gini_a": scores.get(a, {}).get("gini", np.nan),
                    "gini_b": scores.get(b, {}).get("gini", np.nan),
                })

    df_pares = pd.DataFrame(pares)
    if not df_pares.empty:
        df_pares = df_pares.sort_values("asociacion", ascending=False).reset_index(drop=True)
    LOGGER.info("Pares con asociacion > %.2f: %d", cfg.umbral_correlacion, len(df_pares))

    # -----
    # 3.2 Exclusion greedy: gana el de mayor score compuesto
    # -----
    excluidas: dict[str, str] = {} # columna -> motivo
    ganadores: dict[str, str] = {} # columna eliminada -> columna que la desplazo
    decisiones: list[dict] = []

    for _, p in df_pares.iterrows():
        a, b = p["variable_a"], p["variable_b"]
        # Si una ya salio, el par esta resuelto: no se elimina la otra tambien.
        if a in excluidas or b in excluidas:
            decisiones.append({
                "variable_a": a, "variable_b": b, "asociacion": p["asociacion"],

```

```

        "tipo_asociacion": p["tipo_asociacion"],
        "score_a": p["score_a"], "score_b": p["score_b"],
        "variable_conservada": b if a in excluidas else a,
        "variable_eliminada": "-",
        "criterio": "par ya resuelto por una exclusion previa",
    })
    continue

sa = p["score_a"] if pd.notna(p["score_a"]) else -np.inf
sb = p["score_b"] if pd.notna(p["score_b"]) else -np.inf

if sa >= sb:
    gana, pierde, s_gana, s_pierde = a, b, sa, sb
else:
    gana, pierde, s_gana, s_pierde = b, a, sb, sa

criterio = (
    f"score_compuesto {gana}={s_gana:.4f} >= {pierde}={s_pierde:.4f}"
    if np.isfinite(s_gana) and np.isfinite(s_pierde)
    else f"{gana} conservada por score disponible frente a {pierde}"
)
# Desempate exacto: se prefiere el mayor IV y luego el mayor Gini,
# priorizando la metrica que capta relaciones no monotonas.
if np.isfinite(s_gana) and np.isfinite(s_pierde) and np.isclose(s_gana, s_pierde):
    iv_g = scores.get(gana, {}).get("iv", 0) or 0
    iv_p = scores.get(pierde, {}).get("iv", 0) or 0
    if iv_p > iv_g:
        gana, pierde = pierde, gana
    criterio += f" (empate en score; desempate por IV: {gana})"

excluidas[pierde] = (
    f"asociacion {p['asociacion']:.4f} > {cfg.umbral_correlacion} con '{gana}' "
    f"[{p['tipo_asociacion']}]"; {criterio}"
)
ganadores[pierde] = gana
decisiones.append({
    "variable_a": a, "variable_b": b, "asociacion": p["asociacion"],
    "tipo_asociacion": p["tipo_asociacion"],
    "score_a": p["score_a"], "score_b": p["score_b"],
    "variable_conservada": gana, "variable_eliminada": pierde, "criterio": criterio,
})

df_decisiones = pd.DataFrame(decisiones)
LOGGER.info("Variables excluidas por redundancia: %d", len(excluidas))

# -----
# 3.3 VIF sobre las que quedan (solo numericas)
# -----
restantes = [c for c in sobrevivientes if c not in excluidas]
num_restantes = [c for c in restantes if tipos.get(c) in ("NUMERICA", "BOOLEANA")]
vif = pd.Series(dtype=float)
if len(num_restantes) >= 2:
    try:
        vif = calcular_vif(df[num_restantes])
        altos = vif[vif > cfg.umbral_vif]
        LOGGER.info("VIF calculado para %d variables numericas; %d superan %.1f.",
                    len(vif), len(altos), cfg.umbral_vif)
        if cfg.excluir_por_vif and len(altos):
            # Eliminacion iterativa: se quita el VIF mas alto y se recalcula,
            # porque el VIF de una variable depende de las demas presentes.
            trabajo = list(num_restantes)
            while len(trabajo) >= 2:
                v = calcular_vif(df[trabajo])
                if v.empty or v.max() <= cfg.umbral_vif:
                    break
                peor = v.idxmax()
                excluidas[peor] = f"VIF={v.max():.2f} > {cfg.umbral_vif} (colinealidad multiple)"
                trabajo.remove(peor)
                LOGGER.info("    VIF: se excluye '%s' (VIF=%.2f).", peor, v.max())
            vif = calcular_vif(df[trabajo]) if len(trabajo) >= 2 else vif
    except Exception as exc: # noqa: BLE001

```

```

        LOGGER.warning("No fue posible calcular el VIF: %s", exc)

# -----
# Tabla de salida
# -----
filas = []
for col in sobrevivientes:
    fila_asoc = M.loc[col].drop(labels=[col])
    if len(fila_asoc):
        corr_max = float(fila_asoc.max())
        var_corr = str(fila_asoc.idxmax())
    else:
        corr_max, var_corr = np.nan, ""

    info = scores.get(col, {})
    excluida = col in excluidas
    vif_col = float(vif.get(col, np.nan)) if len(vif) else np.nan

    filas.append({
        "columna": col,
        "correlacion_maxima": corr_max,
        "variable_correlacionada": var_corr,
        "tipo_asociacion": _tipo_par(col, var_corr, tipos) if var_corr else "",
        "n_pares_redundantes": int((fila_asoc > cfg.umbral_correlacion).sum()) if len(fila_asoc) else 0,
        "iv": info.get("iv", np.nan),
        "gini": info.get("gini", np.nan),
        "score_compuesto": info.get("score_compuesto", np.nan),
        "vif": vif_col,
        "flg_vif_alto": int(np.isfinite(vif_col) and vif_col > cfg.umbral_vif),
        "flg_exclusion_multivariada": int(excluida),
        "flg_seleccion_final": int(not excluida),
        "variable_que_la_desplaza": ganadores.get(col, ""),
        "motivo_exclusion_multivariada": excluidas.get(col, ""),
        "decision_multivariada": "EXCLUIDA_REDUNDANCIA" if excluida else "SELECCIONADA_FINAL",
    })

multi = pd.DataFrame(filas).sort_values(
    ["flg_seleccion_final", "score_compuesto"], ascending=[False, False]
).reset_index(drop=True)

LOGGER.info("Variables que llegan a la seleccion final: %d de %d.",
            int(multi["flg_seleccion_final"].sum()), len(multi))

# La matriz se devuelve con la columna de nombres como primera columna,
# para que sea legible al volcarla al Excel.
matriz_export = M.copy()
matriz_export.insert(0, "variable", matriz_export.index)
matriz_export = matriz_export.reset_index(drop=True)

return multi, matriz_export, df_decisiones

def obtener_seleccion_final(multi: pd.DataFrame) -> list[str]:
    """Variables que superan las tres fases obligatorias."""
    if multi.empty:
        return []
    return multi.loc[multi["flg_seleccion_final"] == 1, "columna"].tolist()

```